KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

합치는 방법, 눈으로 비교하기

fast-forward, merge, rebase, squash

```
$ git merge 브랜치
  이름표만 앞으로, 또는 합치기 커밋
$ git rebase main
  커밋을 뒤로 옮겨 붙이기
$ git merge --squash 브랜치
  여러 커밋을 하나로
```

오늘의 순서

1

갈라지지 않았을 때

fast-forward, --no-ff

5–6쪽

2

갈라졌을 때

merge, rebase, squash

7–14쪽

3

한눈에 비교

네 가지 결과 그림과 표

15–16쪽

4

git pull은 무엇을 하나

pull 설정 세 가지

17–22쪽

5

GitHub PR의 Merge 버튼

버튼 세 개와 명령

23–25쪽

6

충돌과 주의

충돌, push한 브랜치

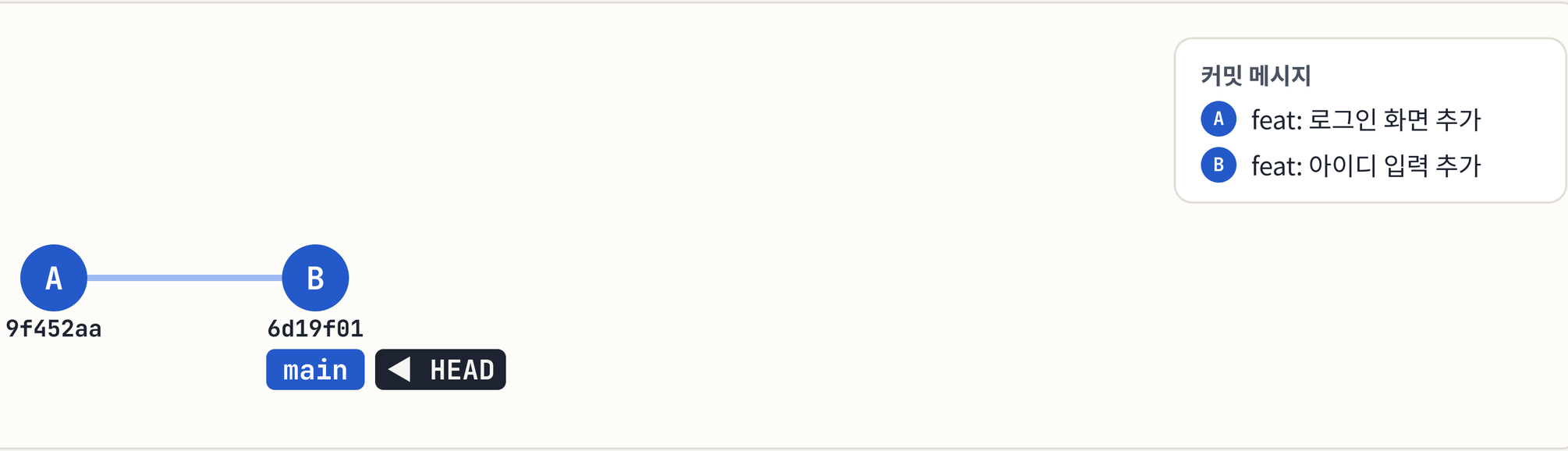
26–29쪽

3–4쪽은 예시 소개, 30–31쪽은 막힐 때와 한 장 요약이에요.

같은 예시 하나로 비교해요

A, B까지 main에 있어요

```
$ git log --oneline
6d19f01 (HEAD -> main) feat: 아이디 입력 추가
9f452aa feat: 로그인 화면 추가
```

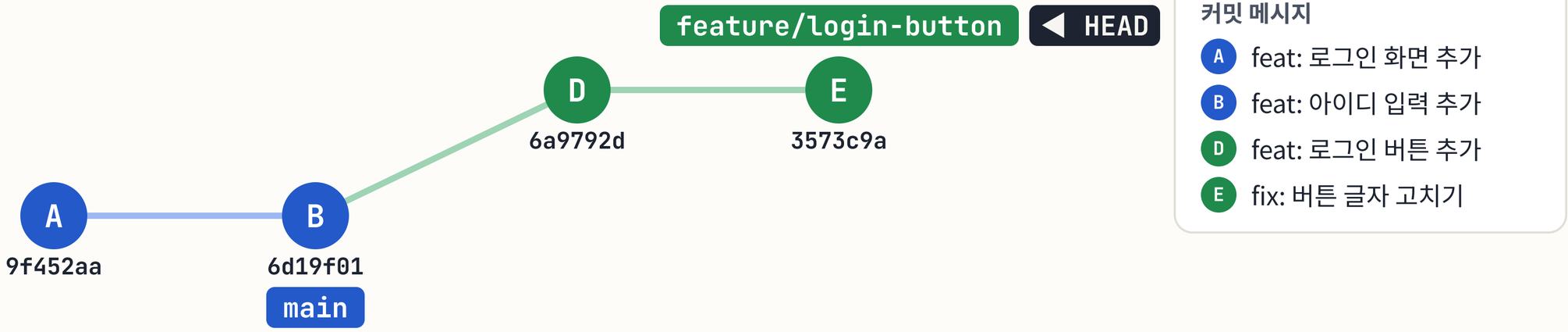


점 안의 글자가 커밋 이름이에요. 아래 글자는 해시예요.

같은 예시 하나로 비교해요

브랜치에서 D, E를 커밋해요

```
$ git checkout -b feature/login-button
$ git commit -m "feat: 로그인 버튼 추가"
[feature/login-button 6a9792d] feat: 로그인 버튼 추가
$ git commit -m "fix: 버튼 글자 고치기"
[feature/login-button 3573c9a] fix: 버튼 글자 고치기
```

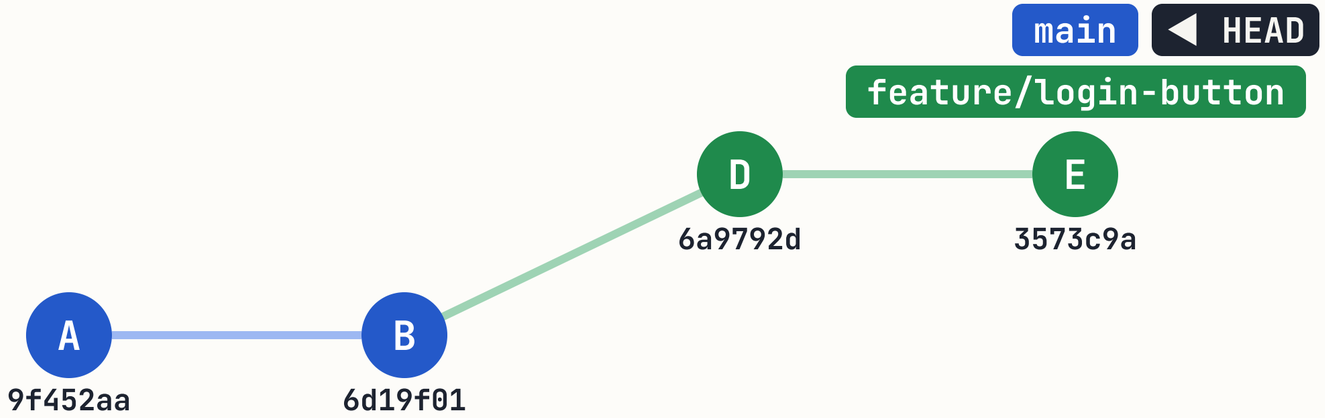


로그인 버튼을 만들고, 글자를 한 번 더 고쳤어요.

1. 갈라지지 않았을 때

fast-forward: main 이름표만 앞으로 가요

```
$ git checkout main
$ git merge feature/login-button
Updating 6d19f01..3573c9a
Fast-forward
```

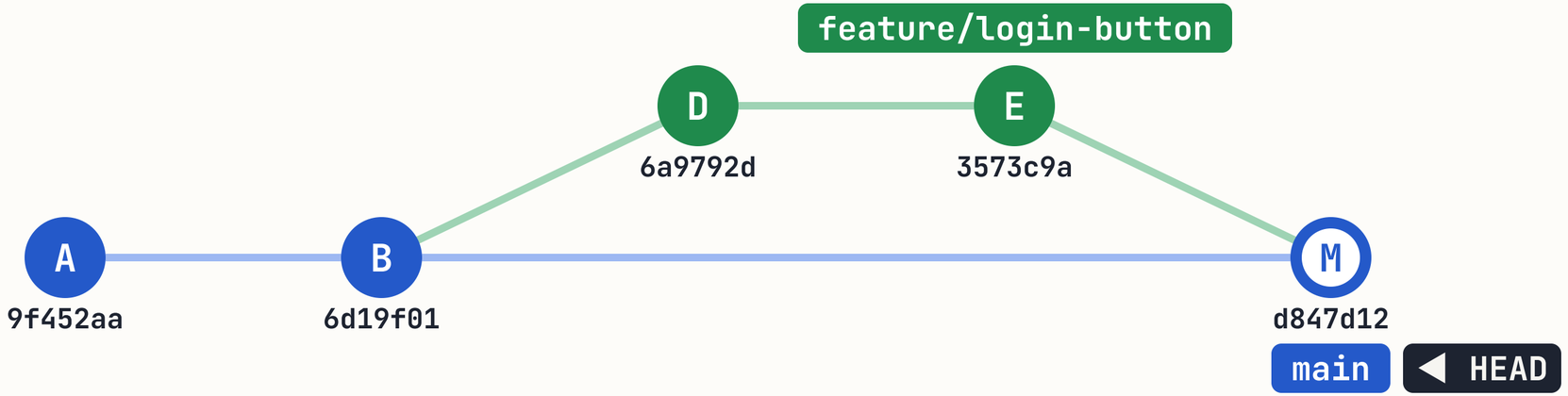


main 쪽에 새 커밋이 없으면 합치기 커밋 없이 끝나요.

1. 갈라지지 않았을 때

--no-ff: 그래도 합치기 커밋을 만들어요

```
$ git merge --no-ff feature/login-button
Merge made by the 'ort' strategy.
$ git log --oneline
d847d12 (HEAD -> main) Merge branch 'feature/login-button'
```

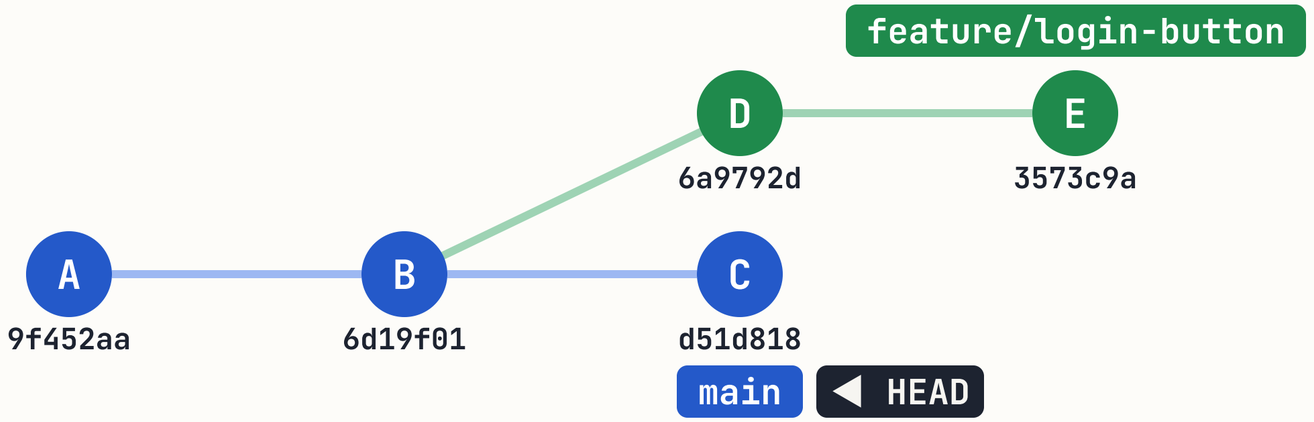


브랜치에서 한 일이 한 덩어리로 보여요. GitHub PR 버튼도 이렇게 합쳐요.

2. 갈라졌을 때

main에 C가 생기면 길이 갈라져요

```
$ git add README.md
$ git commit -m "docs: README 추가"
[main d51d818] docs: README 추가
```



커밋 메시지

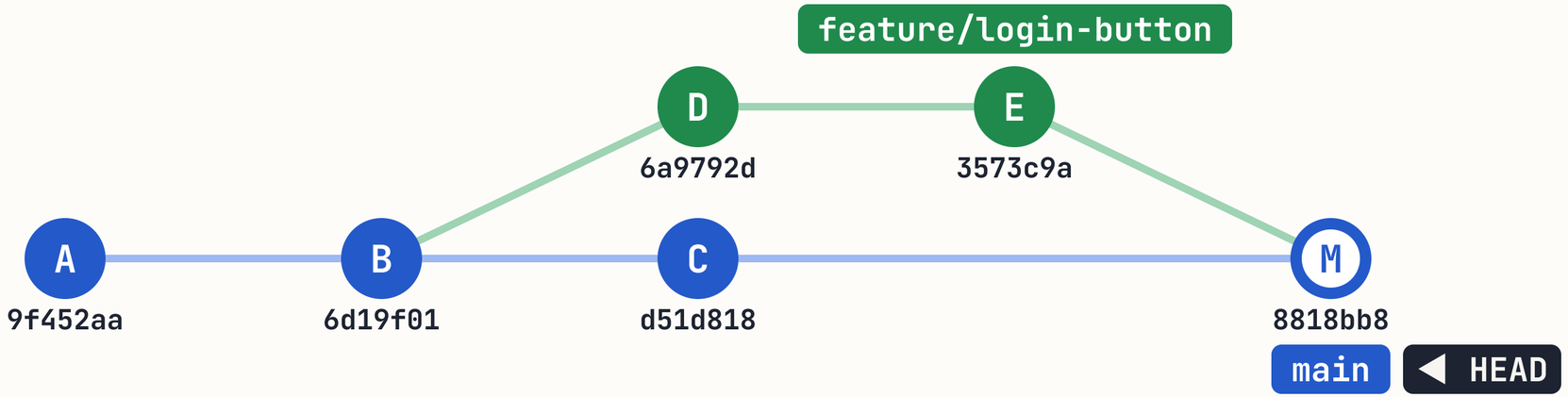
- A feat: 로그인 화면 추가
- B feat: 아이디 입력 추가
- C docs: README 추가
- D feat: 로그인 버튼 추가
- E fix: 버튼 글자 고치기

이제 이름표만 옮겨서는 C와 D, E를 같이 담을 수 없어요.

2. 갈라졌을 때

merge: 두 길을 잇는 합치기 커밋이 생겨요

```
$ git merge feature/login-button
Merge made by the 'ort' strategy.
$ git log --oneline
8818bb8 (HEAD -> main) Merge branch 'feature/login-button'
```

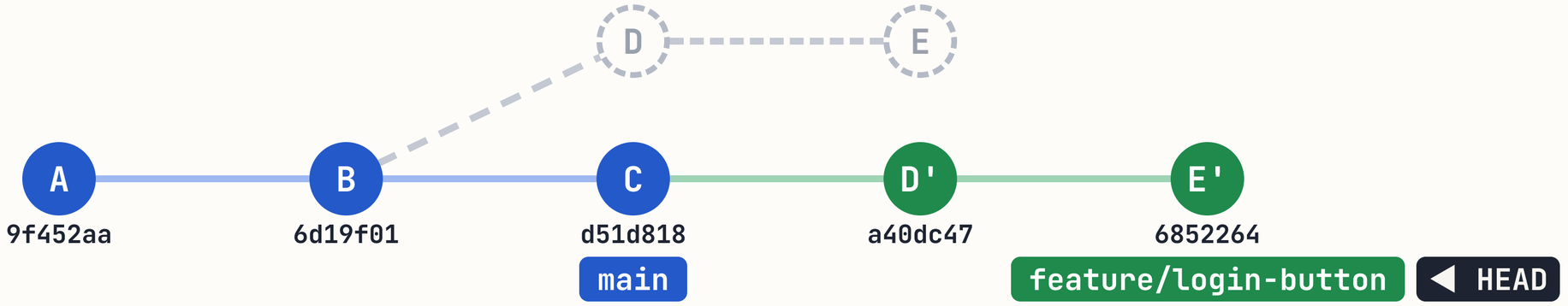


M의 부모는 C와 E 둘이에요. D, E 해시는 그대로예요.

2. 갈라졌을 때

rebase: D, E를 C 뒤로 옮겨 붙여요

```
$ git checkout feature/login-button
$ git rebase main
Successfully rebased and updated refs/heads/feature/login-button.
```



옛 D, E는 점선으로 남고, 같은 내용의 새 커밋 D', E'가 생겨요.

2. 갈라졌을 때

rebase하면 커밋 해시가 바뀌어요

rebase 전

```
$ git log --oneline
3573c9a (HEAD -> feature/login-button) fix: 버튼 글자 고치기
6a9792d feat: 로그인 버튼 추가
6d19f01 feat: 아이디 입력 추가
9f452aa feat: 로그인 화면 추가
```

rebase 뒤

```
$ git log --oneline
6852264 (HEAD -> feature/login-button) fix: 버튼 글자 고치기
a40dc47 feat: 로그인 버튼 추가
d51d818 (main) docs: README 추가
6d19f01 feat: 아이디 입력 추가
```

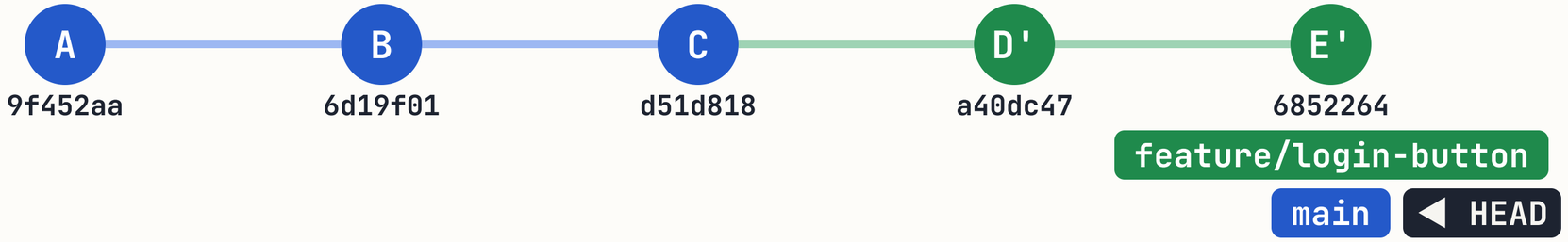
D	~~6a9792d~~	→	D' a40dc47	메시지와 바뀐 내용은 같아요
E	~~3573c9a~~	→	E' 6852264	메시지와 바뀐 내용은 같아요

D의 부모가 B에서 C로 바뀌어서, 뒤에 붙은 E까지 새 커밋이 돼요.

2. 갈라졌을 때

main에서 merge하면 fast-forward로 끝나요

```
$ git checkout main
$ git merge feature/login-button
Updating d51d818..6852264
Fast-forward
```

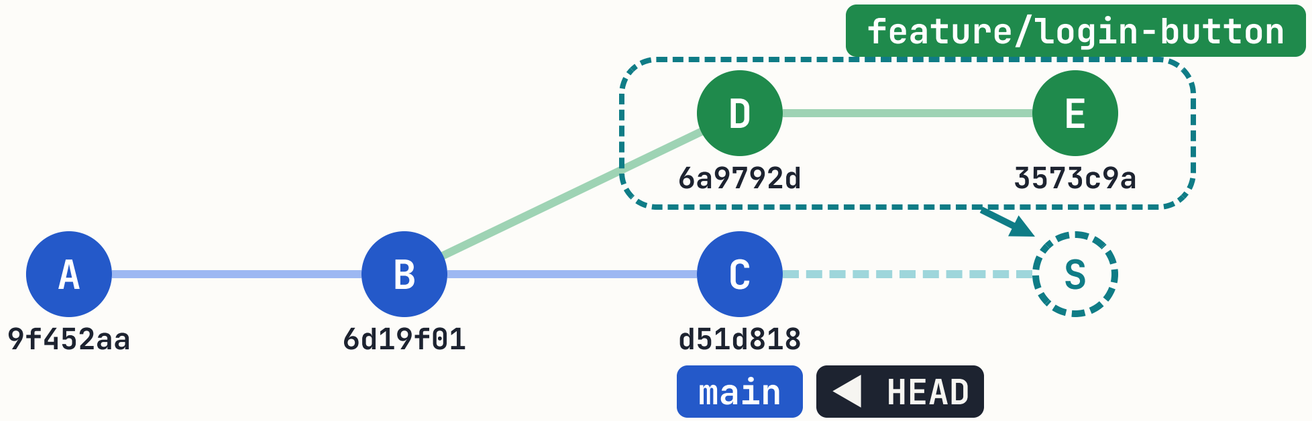


합치기 커밋 없이 기록이 한 줄이 됐어요.

2. 갈라졌을 때

squash: D, E의 변경을 모아 대기실에 올려요

```
$ git merge --squash feature/login-button
Automatic merge went well; stopped before committing as requested
Squash commit -- not updating HEAD
```

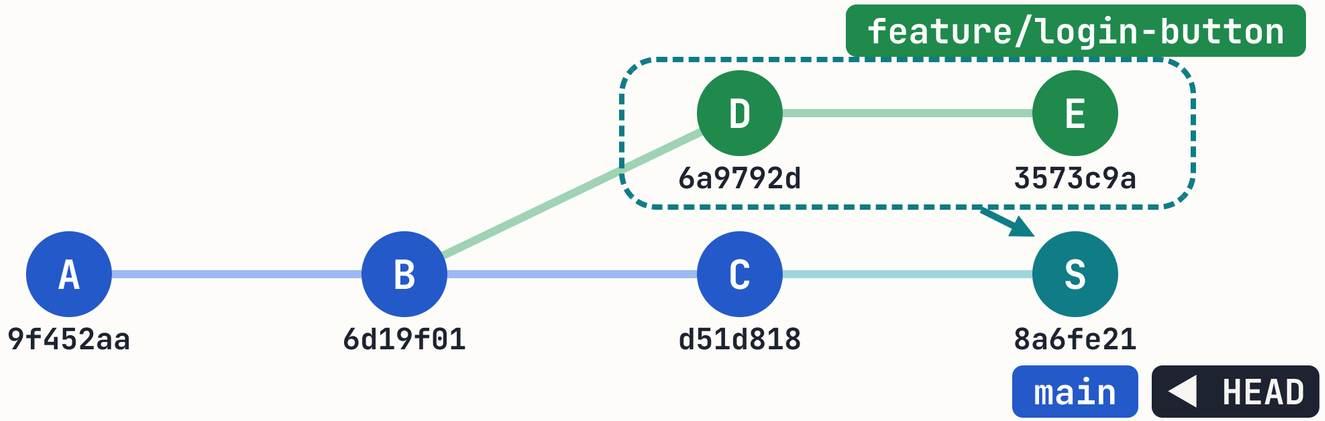


아직 커밋 전이에요. git status에 Changes to be committed로 보여요.

2. 갈라졌을 때

커밋하면 S 하나가 생겨요

```
$ git commit -m "feat: 로그인 버튼 추가"
[main 8a6fe21] feat: 로그인 버튼 추가
```

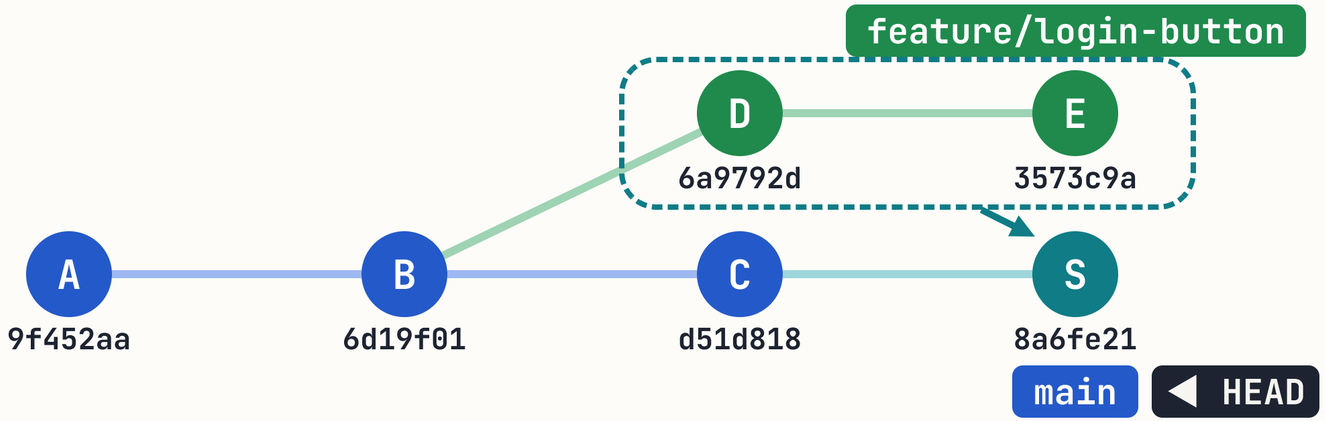


S의 부모는 C 하나예요. 브랜치의 D, E와 선으로 이어지지 않아요.

2. 갈라졌을 때

squash 뒤에는 브랜치가 -d로 안 지워져요

```
$ git branch -d feature/login-button
error: the branch 'feature/login-button' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature/login-button'
```



Git은 D, E가 main에 없다고 봐요. 내용이 들어간 걸 확인했으면 -D로 지워요.

3. 한눈에 비교

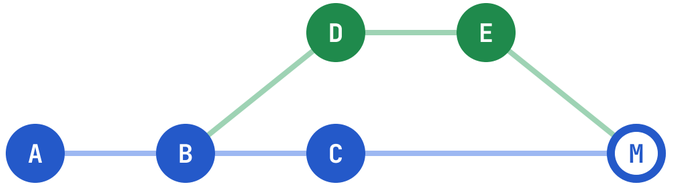
네 가지 결과를 나란히 보면

fast-forward 갈라지지 않았을 때



새 커밋 없음, 이름표만 이동

merge --no-ff도 합치기 커밋



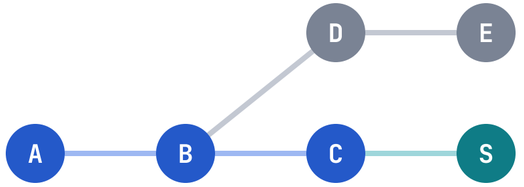
합치기 커밋 M, D, E 그대로

rebase 옮겨 붙인 뒤 fast-forward



한 줄, D', E' 해시가 새로 생김

squash 변경을 모아 커밋 하나로



한 줄, main에는 S 하나만

3. 한눈에 비교

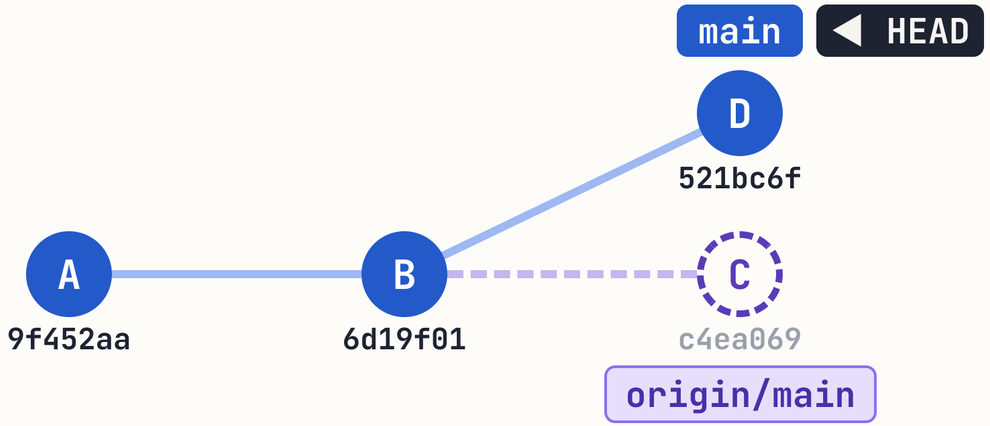
표로 다시 보기

방법	명령	새로 생기는 커밋	D, E 해시	main 기록
fast-forward	`git merge 브랜치`	없음	그대로	한 줄
merge	`git merge 브랜치` `git merge --no-ff 브랜치`	합치기 커밋 M	그대로	두 줄이 만나요
rebase	`git rebase main` `(브랜치에서)`	D', E'	**바뀌어요**	한 줄
squash	`git merge --squash 브랜치` `git commit -m "..."`	S 하나	**main에 없어요**	한 줄, 커밋 하나

4. git pull은 무엇을 하나

pull도 합치기예요

```
$ git fetch origin
$ git log --oneline
521bc6f (HEAD -> main) feat: 로그인 버튼 추가
$ git log --oneline origin/main
c4ea069 (origin/main) docs: README 추가
```



GitHub에는 lee의 C가, 내 main에는 D가 있어요. 둘이 갈라졌어요.

4. git pull은 무엇을 하나

설정이 없으면 pull이 멈추고 세 가지를 알려 줘요

```
$ git pull origin main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
fatal: Need to specify how to reconcile divergent branches.
```

pull.rebase false

merge로 합쳐요

수업 설정

pull.rebase true

rebase로 옮겨 붙여요

보기만 해요

pull.ff only

fast-forward만 받아요

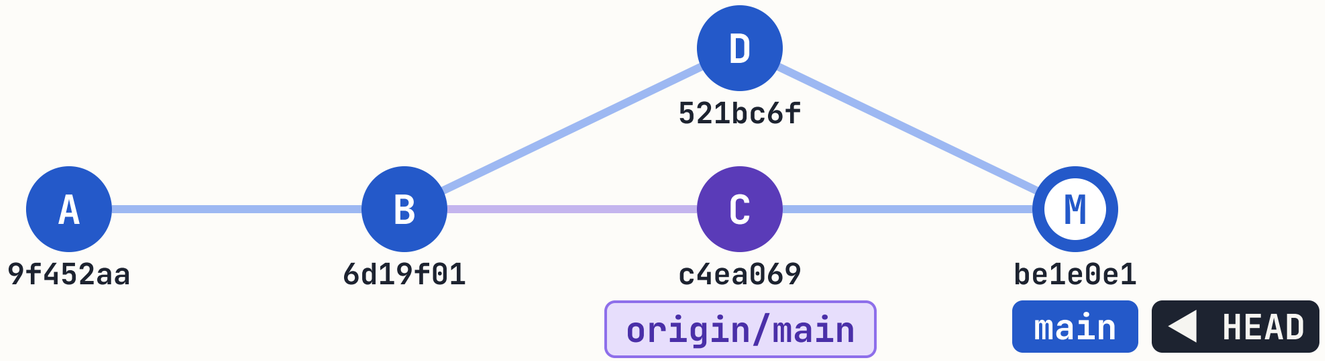
보기만 해요

세 줄이 곧 이 덱에서 본 합치는 방법이에요.

4. git pull은 무엇을 하나

pull.rebase false: 합치기 커밋으로 합쳐요

```
$ git config --global pull.rebase
false
$ git pull origin main
Merge made by the 'ort' strategy.
```

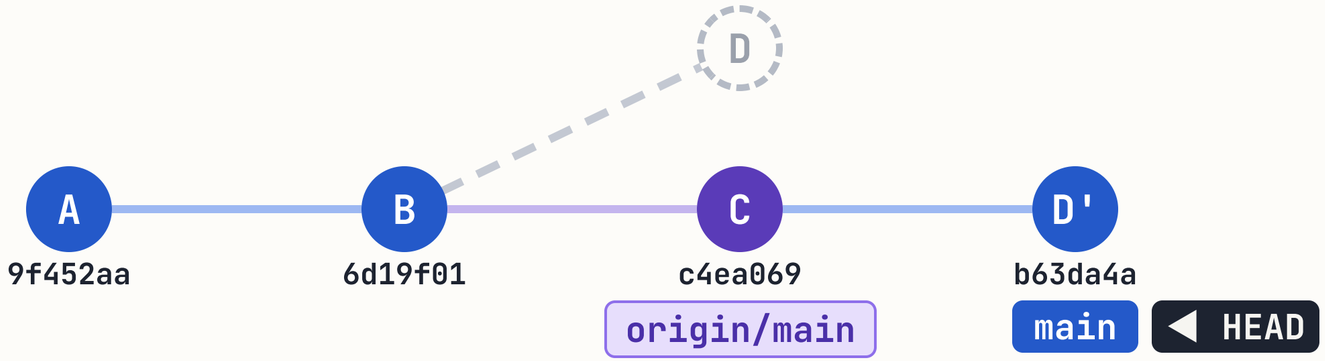


내 D 해시가 그대로예요. 06에서 본 모양이에요.

4. git pull은 무엇을 하나

pull.rebase true: 내 커밋을 C 뒤로 옮겨요

```
# pull.rebase true인 컴퓨터
$ git pull origin main
Successfully rebased and updated refs/heads/main.
$ git log --oneline
b63da4a (HEAD -> main) feat: 로그인 버튼 추가
```

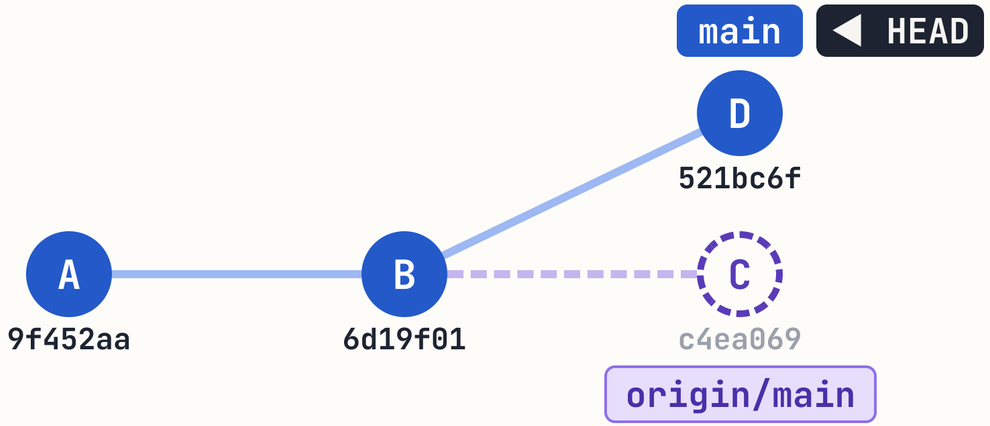


기록은 한 줄이지만 D의 해시가 바뀌었어요.

4. git pull은 무엇을 하나

pull.ff only: 갈라졌으면 받지 않아요

```
# pull.ff only인 컴퓨터
$ git pull origin main
fatal: Not possible to fast-forward, aborting.
```



갈라지지 않았을 때만 fast-forward로 받아요. 갈라지면 직접 골라야 해요.

4. git pull은 무엇을 하나

수업은 pull.rebase false로 통일해요

내 커밋 해시가 그대로예요

push한 커밋과 어긋나지 않아요

합친 자리가 기록에 남아요

합치기 커밋 M이 언제 합쳤는지 보여 줘요

충돌을 푸는 방법이 하나예요

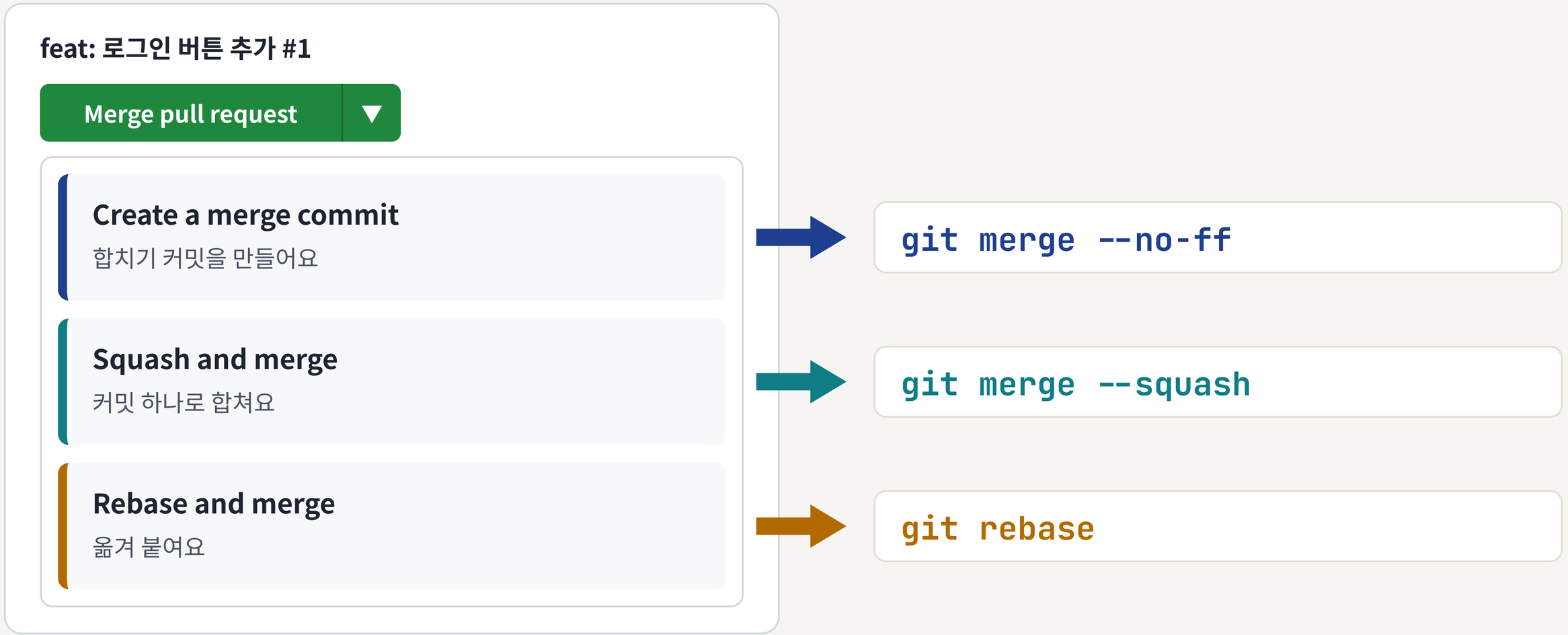
03, 06처럼 고치고 add, commit

```
$ git config --global pull.rebase false
```

06에서 처음 한 번 설정했으면 그대로 둬요.

5. GitHub PR의 Merge 버튼

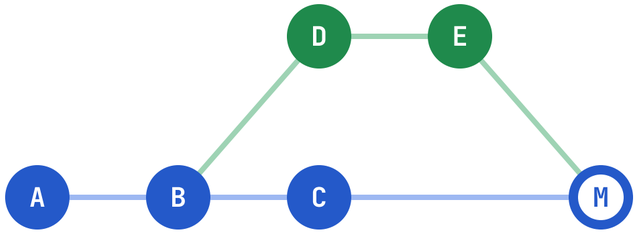
PR의 Merge 버튼은 세 가지예요



5. GitHub PR의 Merge 버튼

버튼마다 GitHub의 main 기록이 달라요

Create a merge commit




```
Merge pull request #1 from
kim/feature/login-button
```

Squash and merge




```
feat: 로그인 버튼 추가 (#1)
```

Rebase and merge




```
D', E' 해시가 새로 생겨요
```

어느 버튼을 쓸지는 팀이 정해요. 정한 버튼 하나만 써요.

5. GitHub PR의 Merge 버튼

Squash나 Rebase 버튼 뒤에는 -D로 지워요

```
$ git checkout main
$ git pull origin main
Updating d51d818..d6514f3
Fast-forward
$ git log --oneline
d6514f3 (HEAD -> main, origin/main) feat: 로그인 버튼 추가 (#1)
$ git branch -d feature/login-button
error: the branch 'feature/login-button' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature/login-button'
```

GitHub에서 PR이 Merged로 바뀐 걸 확인했으면 지워도 돼요.

`git branch -D feature/login-button`

6. 충돌과 주의

충돌은 어느 방법이든 같은 줄이면 나요

main의 login.md

```
# 로그인 페이지
아이디
```

feature/login-title의 login.md

```
# 로그인하기
아이디
```

```
$ git merge feature/login-title
CONFLICT (content): Merge conflict in login.md
```

```
$ git rebase main
CONFLICT (content): Merge conflict in login.md
```

```
$ git merge --squash feature/login-title
CONFLICT (content): Merge conflict in login.md
```

합치는 방법은 기록 모양만 바꿔요. 같은 줄을 다르게 고쳤으면 Git은 늘 물어봐요.

6. 충돌과 주의

rebase 중 충돌은 add 뒤 --continue로 이어 가요

```
$ cat login.md
<<<<<<< HEAD
# 로그인 페이지
=======
# 로그인하기
>>>>>>> bf59624 (feat: 로그인 제목 바꾸기)
아이디
```

1. 표시 세 줄을 지우고 남길 내용으로 고쳐요
2. `git add login.md`
3. `git rebase --continue`

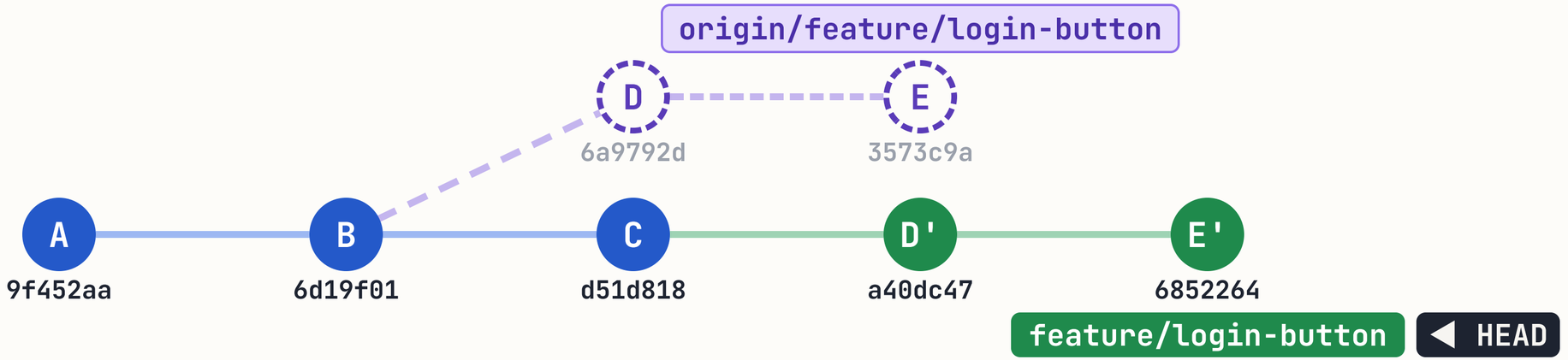
그만두기 `git rebase --abort`

내 브랜치에서 쳤어도 HEAD 쪽은 main 내용이에요. 내 커밋은 아래쪽이에요.

6. 충돌과 주의

push한 브랜치는 rebase하지 않아요

```
$ git rebase main
$ git push origin feature/login-button
 ! [rejected]        feature/login-button -> feature/login-button (non-fast-forward)
```



GitHub에는 옛 D, E가 있어요. 여기서 pull하면 같은 커밋이 기록에 두 번 들어가요.

6. 충돌과 주의

언제 무엇을 쓰나요

git pull 할 때	→	설정대로 merge	`pull.rebase false`
PR을 합칠 때	→	팀이 정한 Merge 버튼 하나	`GitHub`
push 전, 내 브랜치를 최신 main 뒤로	→	rebase를 써도 돼요	`git rebase main`
push한 브랜치, main	→	rebase하지 않아요	`merge만`

막힐 때

이런 메시지가 나와요

1	`fatal: Need to specify how to reconcile divergent branches.` pull 설정이 없어요	`git config --global pull.rebase false`
2	`fatal: Not possible to fast-forward, aborting.` pull.ff only가 설정돼 있어요	`git config --global --unset pull.ff`
3	`error: the branch '...' is not fully merged.` squash나 rebase로 합쳤어요. 합친 걸 확인하고	`git branch -D 브랜치`
4	`error: could not apply ... (rebase 중 CONFLICT)` 고치고 add 뒤 이어 가요. 그만두려면 --abort	`git rebase --continue`
5	`! [rejected] ... (non-fast-forward) rebase 뒤` push한 브랜치를 rebase했어요. pull하지 말고	강사에게 물어봐요

한 장 요약

fast-forward

```
git merge 브랜치
```

갈라지지 않았을 때

이름표만 앞으로

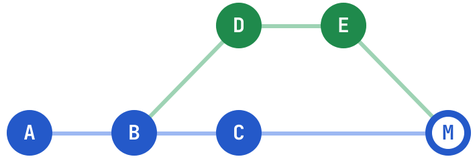


merge

```
git merge 브랜치
git merge --no-ff 브랜치
```

합치기 커밋 M

해시 그대로



rebase

```
git rebase main
```

옮겨 붙이기, 해시가 바뀜

push 전 내 브랜치에만

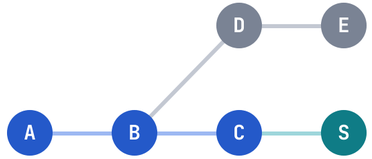


squash

```
git merge --squash 브랜치
git commit -m "..."
```

커밋 하나로

브랜치는 -D로 지우기



pull은 pull.rebase false로 merge, PR은 팀이 정한 버튼, 충돌은 어느 방법이든 같은 줄에서 나요.